ZIG WEB FRAMEWORK

Akamata

A small, fast web framework for Zig 0.16 — explicit, portable, and ready for production workloads.

HTTP · WebSocket · SQLite · D1 · Turso · native · Workers · Containers

THE IDEA

One codebase. Three runtimes.

Keep application code stable while choosing the runtime that fits the workload.

Native

Local development and production servers with SQLite, SSE, and native WebSockets.

Cloudflare Workers

WASM entry points, D1, JSPI, and Durable Object WebSockets.

Cloudflare Containers

Container deployment for workloads that need a longer-lived native process.

Explicit boundaries

Runtime-specific entry points stay visible; handler code stays portable.

DEVELOPER EXPERIENCE

Zig-native from the first handler

```
const State = struct {};
const Ctx = am.Context(State);

fn hello(c: *Ctx) !void {
    try c.text("Hello, Akamata!");
}

_ = try app.get("/", hello);
```

Typed application state

App, Context, request parsing, response helpers, middleware, models, and repositories are parameterized by your state type.

Explicit ownership

Request data lives in the request arena. Errors remain Zig errors. No hidden runtime magic.

DATABASE PORTABILITY

One facade, different terrain

SQLite

Bundled native SQLite for local-first development and single-process services.

D1

Cloudflare binding through the JSPI bridge; one suspend per statement execution.

Turso / libSQL

HTTP pipeline access for remote and multi-region workloads.

Models + migrations

Typed schemas, repositories, versioned migration files, and startup helpers.

The handler changes less than the deployment URL.

OBSERVABILITY

See where the request spends its time

```
GET /api/news
total        43.8 ms
├─ framework  1.2 ms
├─ middleware 0.3 ms
├─ db         38.7 ms
└─ serialize   0.6 ms
```

Request-scoped timing

Monotonic clocks, request IDs, route templates, and lightweight nested spans.

Production-friendly output

Prometheus metrics, structured access logs, and opt-in Server-Timing.

Practical building blocks

- **Routing:** typed params, queries, JSON, form and multipart parsing
- **Middleware:** request IDs, auth, sessions, CSRF, CORS, rate limits, secure headers
- **Realtime:** native WebSockets and SSE; Workers Durable Object integration
- **Outbound work:** HTTP client instrumentation without high-cardinality labels

The framework stays small; application policy remains yours.

A SAFE STARTING POINT

From install to first request

```
git clone https://github.com/appleuser634/Akamata.git
cd Akamata && ./scripts/install.sh
cd ~/projects
akamata init myapp --target=both
cd myapp
zig build run
```

Scaffolded for learning

Note model, CRUD routes, health endpoint, migrations, Workers glue, and deployment files.

Ready to inspect

Read the generated source, replace the state, and keep the same handler shape across backends.

ZIG WEB FRAMEWORK

A focused foundation for web applications.

Zig speed and explicitness, with practical APIs for routing, data, deployment, and observability.

github.com/appleuser634/Akamata · Zig 0.16.x · v0.0.1